

Robot Cat

Uzupełnienie: montaż, architektura, kompute, zasilanie

Ten dokument uzupełnia napedy-v4.pdf, plan-zakupowy.pdf i montaz.pdf notatkami z rozmowy, które nie miały jeszcze gdzie trafić: dlaczego montaż napędów jest w stawie, nie w brzuchu; dlaczego kompute jest jeden centralny, nie per-noga; porównanie z napędami QDD; i uczciwa ocena płynności chodu. Sekcje 1-8 to skrót ustaleń opisanych szerzej w napedy-v4.pdf — zachowane dla ciągłości, nie duplikują decyzji.

Wniosek w jednym zdaniu

Kot chodzi 0,14 m/s i to jest sufit serw pozycyjnych klasy hobby, nie wynik słabego strojenia. Różnica między chodzeniem a bieganiem to nie kwestia wymiarów ani parametrów chodu, tylko wyłącznie klasy napędu.

1. Stan projektu

Symulacja jest dopracowana i nie jest wąskim gardłem. Model ma proporcje prawdziwego kota, chód jest strojony pomiarowo, a cały stos ma pokrycie testami i CI.

Parametr	Wartość	Uwaga
Stopnie swobody	12 (4 nogi × 3)	biodro 2 DOF + kolano
Masa całkowita	2,10 kg	korpus 1,04 kg + nogi po 234 g
Segment nogi	0,11 + 0,11 m	zasięg 0,22 m
Wysokość w kłębie	24,2 cm	biodro 17,2 cm, rozstaw bioder 22 cm
Sylwetka (kłąb / rozstaw)	1,10	kot ≈ 1,0 (kwadratowa)
Prędkość w symulacji	0,14 m/s	na limitach ST3215
Kołysanie boczne	1,6°	międzyszczytowo
Dryf kursu	≈0,9°/m	chód otwarty, bez sprzężenia
Testy	581, CI zielone	matematyka bez symulatora

2. Symulacja ma limity kupionego serwa

Model deklaruje w każdym przegubie nóg moment 2,57 Nm i prędkość 4,7 rad/s — to karta katalogowa ST3215 przy najgorszym stanie pakietu, nie wartości dobrane tak, żeby chód wyglądał dobrze. Głowa i ogon dostają 0,20 Nm mikroserwa. Dzięki temu każda godzina strojenia chodu dotyczy robota, który da się zbudować.

Uwaga: sam limit w modelu nie wystarczy

Gazebo nie egzekwuje limitu prędkości przy sterowaniu pozycyjnym. Po ustawieniu limitu 3,5 rad/s przeguby i tak kręciły 10 rad/s. Wpisana wyżej wartość 4,7 rad/s dokumentuje więc napęd, ale nie chroni przed zadaniem mu czegoś niewykonalnego — uczciwy pomiar wymaga ograniczenia samego generatora chodu, dokładnie tak, jak zachowa się prawdziwy sterownik.

3. Co naprawdę ogranicza prędkość

Szczytowa prędkość przegubu w ustalonym chodzie, policzona dla siatki parametrów. Kolumna „limit” to prędkość, jaką chód może maksymalnie nadać ciału.

Krok [m]	Cykl [s]	Limit [m/s]	Szczyt [rad/s]	Serwo ~4,8 rad/s
0,16	0,30	0,533	24,6	nie
0,12	0,40	0,300	14,8	nie
0,08	0,50	0,160	9,8	nie
0,06	0,50	0,120	9,1	nie
0,04	0,60	0,067	7,2	nie

Kluczowe odkrycie

Nawet najwolniejszy chód z domyślnym wymachem żąda 7,2 rad/s — półtora raza więcej, niż serwo potrafi. O szczycie decyduje więc unoszenie łapy, a nie długość kroku. Skrócenie kroku samo z siebie nie pomaga; trzeba obniżyć wymach.

Chód wykonalny na serwach

Po dopuszczeniu wymachu jako trzeciego parametru znaleziono rozwiązania mieszczące się poniżej 3,4 rad/s — z zapasem na obciążenie, bo katalogowe 4,8 rad/s dotyczy biegu jałowego.

Krok [m]	Cykl [s]	Wymach [m]	Szczyt [rad/s]	Limit [m/s]
0,06	0,6	0,012	3,1	0,100
0,08	0,8	0,012	2,8	0,100
0,08	1,0	0,012	2,3	0,080

Wiersz wyróżniony zmierzono następnie w pełnej symulacji: 0,143 m/s osiągnięte (3,42 m w 24 s czasu symulacji).

4. Moment — druga granica

Momenty odczytane z interfejsu stanu, przy masie 2,10 kg. Pełne wyliczenie budżetu masy i doboru napędu jest w napedy-v4.pdf — tu tylko liczby, do których odwołują się dalsze sekcje.

Stan	Moment na przegub	Uwaga
Leży	0,09 Nm	praktycznie bez obciążenia
Stoi	0,72 Nm	obciążenie ciągłe
Idzie 0,1 m/s — mediana	0,18 Nm	przez większość czasu
Idzie 0,1 m/s — 95. pct	1,93 Nm	liczba do doboru napędu

Dlaczego 95. percentyl, a nie szczyt

Chwilowe szczyty to transjenty kontaktowe — uderzenie łapy o podłoże, nie zapotrzebowanie ustalone. W prawdziwym robocie amortyzuje je podatność mechaniczna, czyli miękka nakładka łapy; dlatego montaz.pdf traktuje ją jako element nośny, a nie ozdobę. Liczbą do doboru napędu jest 95. percentyl: 1,93 Nm wobec 2,57 Nm, jakie ST3215 daje na rozładowanym pakiecie.

5. Porównanie klas napędów

Kolumna „kot w symulacji” to prędkość zmierzona po nałożeniu ograniczeń danej klasy — nie przeliczenie z karty katalogowej.

	ST3215 (wybrane)	Feetech STS3250	Dynamixel XM430	CubeMars AK60-6
Prędkość	~4,7 rad/s	~7,9 rad/s	~4,8 rad/s	~30–60 rad/s
Moment (utyk / ciągły)	2,9 / ~0,7 Nm	4,9 / ~2,45 Nm	4,1 / ~1,3 Nm	9 / 3 Nm
Masa ×12	0,66 kg	0,89 kg	1,0 kg	4,4 kg
Kot w symulacji	0,14 m/s	0,14 m/s	0,14 m/s	0,58–0,64 m/s
Bieg (faza lotu)	niemożliwy	niemożliwy	niemożliwy	realny
Koszt 12 szt.	~1 tys. zł	~1,5–2 tys. zł	~12 tys. zł	~6–7 tys. zł
Klasa odniesienia	SpotMicro, Pupper	SpotMicro, wersja o wyższym momencie	roboty badawcze	MIT Mini Cheetah

Najważniejsze w tej tabeli: Feetech i Dynamixel leżą w tej samej lidze prędkości. Dynamixel kupuje niezawodność, moment i wsparcie — nie szybszego kota. Skok jakościowy pojawia się dopiero przy napędach bezszczotkowych quasi-direct-drive, i jest to skok o rząd wielkości.

Decyzja: Feetech ST3215

Wymaganie to 1,93 Nm w 95. percentylu przy masie 2,10 kg — pełne wyliczenie w napedy-v4.pdf. ST3215 daje 2,94 Nm utyku przy 12 V i 2,57 Nm przy rozładowanym pakiecie 3S, czyli 33% zapasu w najgorszym punkcie. Mocniejsze serwa z tej tabeli kupują zapas, którego nie ma czym wykorzystać: prędkość kota ogranicza nie moment, tylko quasi-statyczny charakter chodu.

Zastrzeżenie do sprawdzenia na sprzęcie: przy długo utrzymywanym, unieruchomionym obciążeniu zabezpieczenie termiczne serw tej klasy ścina moment. Dotyczy to stania w

miejscu przez dłuższy czas oraz póź stretch/lie-down — nie samego chodu, gdzie 95. percentyl to krótkie, powtarzalne szczyty. Stąd tryb leżenia jest w projekcie także chłodzeniem: leżący kot obciąża przeguby momentem 0,09 Nm wobec 0,72 Nm w staniu.

Źródło: servodatabase.com oraz karta katalogowa [Waveshare ST3215](#) (30 kg·cm przy 12 V).

6. Czy ten kot może biegać?

Kryterium przejścia między chodami u zwierząt opisuje liczba Froude’a: kłus przechodzi w galop przy $Fr \approx 2,5$, gdzie $Fr = v^2 / (g \cdot L)$, a L to długość nogi. Dla naszej nogi 0,16 m daje to próg galopu:

$$v = \sqrt{(2,5 \times 9,81 \times 0,16)} \approx 2,0 \text{ m/s}$$

Wymiary nie są przeszkodą — 2,0 m/s to dokładnie tempo, w jakim galopują zwierzęta tej wielkości. Natura biega w tej skali bez trudu. Przeszkody są trzy i żadna nie dotyczy geometrii:

Przeszkoda	Dlaczego	Status
Moc napędu	bieg wymaga fazy lotu, czyli impulsu większego niż ciężar — momentu i prędkości jednocześnie	tylko QDD
Sprzężenie zwrotne	lądowania nie da się wykonać na ślepo; potrzebny co najmniej IMU	do zrobienia
Wzorec chodu	galop to inne przesunięcia fazowe i kontrola pochylenia — czysta matematyka	w zasięgu dziś

Warto zauważyć: wzorec galopu można prototypować w symulacji już teraz — podnosząc limity w modelu do klasy QDD — i mieć go gotowego, zanim takie silniki zostaną kupione.

Dlaczego nie od razu QDD?

	ST3215 (wybrane)	CubeMars AK60-6 (QDD)
Koszt, 12 szt.	~1,3 tys. zł	~6-7 tys. zł (5×)
Kot w symulacji	0,14 m/s, chód	0,58-0,64 m/s, bieg realny
Sterowanie	pozycyjne (docelowy kąt)	momentowe/impedancyjne (docelowa siła)
Architektura	jeden Raspberry Pi, 100 Hz	wymaga pętli rzędu 1 kHz — realnie osobny kontroler per noga, patrz sekcja 9

Różnica kosztu (3-4×) to najmniejszy powód. Większy: QDD wymaga innej architektury sterowania, nie tylko innych silników. Sterowanie momentem/impedancją potrzebuje pętli rzędu 1 kHz na przegub — jedno centralne Raspberry Pi tego nie udźwignie (patrz sekcja 9), więc realnie dochodzi sterownik per noga, co samo w sobie jest osobnym projektem elektroniki. Do tego bieg wymaga rzeczy, których jeszcze nie mamy: sprzężenia z lądowania (IMU + regulator, sekcja 10) i wzorca galopu (matematyka gotowa, niewdrożona).

Skok na QDD teraz oznaczałby budowanie wszystkiego naraz — nowej mechaniki, nowego sterowania niskopoziomowego, nowego wzorca chodu — bez zweryfikowania żadnego z osobna. Tanie serwa pozwalają sprawdzić cały tor „symulacja → sprzęt” (mechanika,

okablowanie, interfejs) tanim kosztem błędu: \$15 serwo, nie \$150+ silnik QDD, jeśli coś w montażu jest źle. To nie jest droga bez wyjścia — generator chodu, kinematyka i testy przenoszą się bez zmian na QDD później (sekcja 7); wymiana napędów to wymiana jednego bloku sprzętowego, nie przepisanie projektu od zera.

7. Rekomendacja

Zbudować najpierw wersję na tanich serwach, traktując ją jako weryfikację całego toru „symulacja → sprzęt”: mechaniki, elektroniki, interfejsu sterowania i procesu strojenia. Kot będzie chodził, nie biegał — i to jest akceptowalny wynik pierwszej iteracji.

Kluczowy argument: wymiana napędów nie unieważnia oprogramowania. Architektura przewiduje podmianę samego bloku sprzętowego; generator chodu, kinematyka i testy przenoszą się bez zmian. Przejście na napędy bezszczotkowe to później wymiana silników i ponowne strojenie parametrów — procedura, którą już wykonaliśmy pomiarowo.

Kolejne kroki

	Krok	Po co
1	Chód serwowy jako nazwany profil w repozytorium	gotowy punkt odniesienia dla realnego sprzętu
2	✓ Rozstrzygnięte: Feetech ST3215, patrz sekcja 5	33% zapasu momentu przy masie 2,10 kg
3	✓ Rozstrzygnięte: montaż w stawie, patrz sekcja 8	prosta mechanika na pierwszą iterację
4	✓ Rozstrzygnięte: IMU BNO085 oraz Pi 5 + AI HAT+, patrz sekcje 9-11	IMU usuwa dryf; akcelerator daje rozpoznawanie obrazu w czasie rzeczywistym
5	Prototyp galopu w symulacji	gotowy, zanim pojawią się silniki zdolne go wykonać
6	Interfejs sprzętowy — podmiana bloku hardware	pierwsze uruchomienie fizycznego robota
7	✓ Rozstrzygnięte: zasilanie i ładowanie, patrz sekcja 13	3S LiPo + gniazdo w obudowie, bez wyjmowania baterii
8	Stacja dokująca	odłożone — trudniejsze niż u robota kołowego, patrz sekcja 12

8. Montaż napędów: w stawie, nie w brzuchu

Każda z 4 nóg dostaje 3 identyczne ST3215 w układzie „klastra biodrowego”, odwzorowującym łańcuch z URDF (hip_link → thigh_link → calf_link):

Przegub	Montaż	Oś wyjścia
Biodro / roll	na sztywno do korpusu, w każdym z 4 rogów	równoległa do X (przód-tył)

Przegub	Montaż	Oś wyjścia
Udo / pitch	na wsporniku hip_link — jedzie razem z silnikiem roll	równoległa do Y (lewo-prawo)
Kolano / calf pitch	bezpośrednio w stawie kolanowym, na thigh_link	równoległa do Y (lewo-prawo)

Alternatywą byłby napęd zdalny: silniki w brzuchu, ciągną lub pasek do dalszych przegubów. Jedyna korzyść — mniejsza bezwładność wymachującej nogi — liczy się w dynamicznym chodzie przy prędkości (klus, galop), nie w quasi-statycznym marszu 10 cm/s z sekcji 5. Opłacamy złożoność mechaniczną bez żadnego zysku na tym etapie.

Koszty, których unikamy: luz i rozciągnięcie paska/cięgna występują za enkoderem samego serwa, więc komenda i rzeczywisty kąt kolana rozjeżdżają się, chyba że dojdzie drugi enkoder w stawie; małe paski i kółka pasowe w skali 22 cm to nie część z półki, tylko osobny projekt mechaniczny — dokładnie ten rodzaj złożoności, przed którym ostrzega rekomendacja „pierwsza iteracja na tanich serwach” z sekcji 7.

Napęd zdalny zostawiamy na tę samą przyszłą wymianę, co silniki bezszczotkowe QDD z sekcji 6 (bieg) — przebudowa mechaniki raz, przy okazji zmiany klasy napędu, nie dwa razy.

9. Architektura sterowania: jeden procesor centralny

Rozważaliśmy: procesor per noga czy jeden centralny. Odpowiedź jest jednoznaczna — jeden centralny, i to nie jest uproszczenie na siłę.

Chód wymaga wiedzy o całym ciele naraz: przekątne nogi w przeciwfazie (trot), różnica długości kroku lewo/prawo przy skręcie, balans na czterech łapach. Rozbicie tego na 4 niezależne procesory oznaczałoby synchronizowanie fazy chodu między płytkami — opóźnienia komunikacji i rozjazd zegarów, dokładnie ten problem, którego jedna centralna pętla unika z definicji. Do tego serwa STS już mają własny mikrokontroler robiący pętlę pozycji lokalnie — to już jest „procesor per staw”, wbudowany w silnik. Dodanie kolejnego poziomu (płytki per noga) nic by nie dało ponad to, co serwo robi samo.

Kiedy per-noga zaczyna mieć sens

Dopiero przy przejściu na napędy bezszczotkowe QDD i bieg z fazą lotu (sekcja 6) — tam potrzeba pętli sterowania momentem rzędu tysięcy Hz, gdzie centralne Raspberry Pi fizycznie nie nadąża. To ta sama przyszła wymiana co reszta odłożonych rzeczy, nie dotyczy pierwszej iteracji.

10. Percepcja: co widzi i czuje kot

Element	Wybór	Dlaczego
Kamera	Raspberry Pi Camera HD v3 12MPx (IMX708)	standard 119,90 zł / szeroki kąt 169,00 zł, w magazynie Botland
Odległość / przeszkody	VL53L5CX, ToF 8×8 stref, 63°	daje kąt/mapę, nie jedną liczbę jak pojedynczy ToF czy sonar — 69,90 zł, Pololu

Element	Wybór	Dlaczego
Orientacja	BNO085 (Adafruit 4754)	następca BNO055 (którego Botland nie ma); fuzja sensorów on-chip, 135,00 zł

Czego świadomie nie bierzemy

Bez LiDAR-u — skanuje jedną płaszczyznę na stałej wysokości, a korpus kota buja się w pionie podczas chodu; zła geometria dla czworonoga, do tego ciężki i drogi w tej skali. Bez czujników krawędzi/schodów — środowisko docelowe jest płaskie, ewentualnie nierówne (kamienie), bez schodów. Bez głębi stereo z dwóch „oczu” — wymagałaby synchronizacji dwóch modułów kamery i własnej kalibracji/dopasowania obrazów; VL53L5CX daje tę samą informację prościej. IR/NoIR tylko na żądanie — wariant NoIR kamery i doświetlacz LED 850nm są w katalogu, ale mają sens wyłącznie jeśli kot ma działać po ciemku.

11. Kompute: Raspberry Pi 5 + AI HAT+

Rozstrzygnięcie — kosztem budżetu

Pi 5 8GB (829,90 zł) + AI HAT+ 13 TOPS (329,00 zł) są na liście zakupowej. Powód jest techniczny, nie preferencyjny: Pi 4B nie ma złącza PCIe, więc akcelerator Hailo jest z nim fizycznie niemożliwy — nie gorszy, niemożliwy. Ceną jest budżet: cała lista wychodzi 3788,60 zł wobec pierwotnych 2500 zł, i to jest jedyne miejsce, gdzie ta różnica powstała.

System operacyjny: Raspberry Pi OS (64-bit), nie Ubuntu — mimo że oficjalne pakiety ROS 2 celują w Ubuntu, to Ubuntu nie ma wbudowanej obsługi czujnika Camera Module 3 (IMX708) i wymaga budowania forka libcamera ze źródeł. Raspberry Pi OS ma to od ręki. ROS 2 Jazzy stawiamy przez pixi/RoboStack (ten sam mechanizm co w symulacji na Macu) — RoboStack publikuje gotowe paczki na linux-aarch64, więc nie trzeba oficjalnych pakietów Ubuntu.

Akcelerator to AI HAT+ 13 TOPS — moduł Hailo-8L na oficjalnej płytce M.2, wpinany wprost w złącze PCIe Pi 5. Dystanse i taśma PCIe są zwykle w pudełku.

	YOLOv8n, 640×640
Samo CPU Pi 5	~12 FPS
Z Hailo-8L	~137 FPS

To ~11× przyspieszenie — ale ważniejsza niż liczba klatek jest ta konsekwencja: bez akceleratora cała moc obliczeniowa Pi idzie w widzenie i nic nie zostaje na chód ani resztę węzłów ROS. Z osobnym układem widzenie działa praktycznie za darmo w tle. Jeden moduł wystarcza: 13 TOPS to zapas o rząd wielkości większy, niż potrzebuje jedna kamera przy prędkości 10 cm/s. Kolejność prac to nie zmienia — reaktywne omijanie przeszkód opiera się na ToF i działa, zanim ktokolwiek uruchomi sieć neuronową.

12. Interakcja i dokowanie

Funkcja	Element	Uwaga
Głaskanie	MPR121, 12 stref pojemnościowych, I2C	strefy wzdłuż grzbietu/głowy, nie jedno „dotknięto: tak/nie” — ok. 40-60 zł
Miauczenie	MAX98357A + głośnik	wzmacniacz I2S 3W, 34,90 zł w Botlandzie

Funkcja	Element	Uwaga
Kontroler PS4	Bluetooth wbudowany w Pi + ROS 2 joy/teleop_twist_joy	zero nowego sprzętu — paruje się jak zwykły gamepad, mapuje na to samo /cmd_vel

Stacja dokująca — odłożone

Odkurzacz najeżdża na styki kołami — precyzyjne pozycjonowanie jest tanie mechanicznie. Czworonóg musi trafić stopami w styki z dokładnością chodu, który sam w sobie ma dryf i poślizg — to trudniejszy problem sterowania niż samo dokowanie. Realistyczna droga: styki pogo-pin na piersi/brzuchu + marker wizyjny (ArUco/AprilTag) na stacji do precyzyjnego podejścia na ostatnim odcinku, wykrywany już posiadaną kamerą. To krok po tym, jak chodzenie i nawigacja będą solidne, nie na pierwszą iterację — ten sam porządek co bieganie w sekcji 6.

13. Zasilanie

3S LiPo (11,1V, mieści się w zakresie 6–12,6 V serwa ST3215 bez przetwornicy). Pojemność celowo z zapasem, „im dłużej tym lepiej” — ale to nie jest darmowy wybór: pojemność LiPo skaluje się z wagą niemal liniowo, a cięższy robot podbija z powrotem budżet momentu w stawach z sekcji 4/5. Realny czas pracy trzeba będzie zmierzyć na gotowym sprzęcie, nie zakładać z góry.

Pakiet	Orientacyjny czas ciągłego chodu
3S 2200 mAh	~22-26 min
3S 3000 mAh	~30-36 min
3S 5000 mAh	~50-60 min

Ładowanie bez wyjmowania baterii: gniazdo XT60 (lub XT30) montowane w obudowie, na stałe podłączone do pakietu wraz z wyprowadzonymi pinami balansera — zewnętrzna ładowarka balansująca wpina się z zewnątrz, bateria zostaje w środku. Osobna gałąź 5V (przetwornica) dla Raspberry Pi, odizolowana od zasilania serw, żeby skoki prądu serw nie resetowały Pi.

14. Płynność chodu: uczciwa ocena

Pytanie wprost: czy to będzie chodzić płynnie, jak prawdziwy kot, czy jak „pokraczne” roboty z internetu? Odpowiedź jest pośrodku, i wynika z fizyki wyboru napędu, nie z braku starań.

Trajektorie są już wygładzone i przetestowane (sinusoidalne uniesienie łapy, filtr dolnoprzepustowy na komendach, brak skoków między taktami), a ST3215 ma metalowe przekładnie — to eliminuje szarpanie typowe dla tanich serw hobby z luźnym plastikowym osprzętem. Twardy sufit: to są serwa pozycyjne, nie momentowe. Prawdziwy kot chodzi płynnie, bo jego stawy są podatne — reagują na siłę jak sprężyna, a nie „jedź do kąta X bez względu na kontakt z podłożem”. To różnica strukturalna, nie kwestia strojenia. Do tego chód jest otwarty (bez sprzężenia z kontaktem łapy) — na nierównym podłożu (kamienie) każde odchylenie od planu pokaże się jako widoczna korekta, nie płynna adaptacja.

Prawdziwa kocia gracia wymaga sterowania momentem (impedancyjnego) — to ta sama granica, co przejście na napędy QDD do biegania w sekcji 6. Jeśli płynność ruchu ma wysoki priorytet, to argument za wcześniejszym rozważeniem tej wymiany, nie tylko „żeby biegał” — ale to świadomy kompromis kosztowy, nie coś przy okazji.

Co ten dokument świadomie pomija

Momenty oparto na masach członów z modelu, które są orientacyjne — realna konstrukcja zmieni budżet w obie strony (lżejsze człony, ale dochodzi bateria i elektronika). Ceny są rzędu wielkości. Nie analizowano sztywności ramy, przekładni ani zasilania — na tym etapie decyzja dotyczy klasy napędu i miejsca montażu, nie konkretnego numeru katalogowego wszystkich śrub.